



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: BASES DE DATOS

PRÁCTICA 1: MODELO ENTIDAD-RELACIÓN

Ejercicio 5 — Caso de Estudio Catálogo Musical y Motor de Recomendación por Audio

Profesor: Hurtado Avilés Gabriel

Alumno: Hernandez Barrios Samuel Rodrigo

Tercer Semestre

Grupo: 3CV4

Fecha: 18 Septiembre 2026

Ejercicio 5. Caso de Estudio: Catálogo Musical y Motor de Recomendación por Audio

1. Descripción de la problemática

Las plataformas de streaming de audio actuales suelen construir sus recomendaciones sobre señales superficiales: el género declarado de una canción o su popularidad agregada. Este enfoque produce una experiencia pobre porque ignora el comportamiento real de escucha de cada usuario y las características acústicas objetivas de las pistas (tempo, energía, bailabilidad), tratando por igual a un oyente casual que escucha los primeros quince segundos de una canción y abandona, y a un oyente que la reproduce completa y la agrega a una playlist propia.

El reto de diseño consiste en construir un esquema relacional que centralice el catálogo musical con sus atributos acústicos, capture con precisión el log de reproducciones (qué se escuchó, cuánto tiempo, en qué dispositivo), y sostenga la organización del contenido por parte de los usuarios en playlists propias, de forma que esa información combinada permita generar recomendaciones basadas en similitud de audio y en comportamiento real de escucha, en lugar de depender únicamente del género o la popularidad.

2. Entrevista simulada con el cliente (Gerente de Producto)

Consultor: *Para empezar, ¿qué información necesitan guardar de cada canción, más allá del título y el artista?*

Gerente de Producto: Necesitamos el álbum al que pertenece, la duración exacta, el género, y sobre todo los atributos de audio que hoy no estamos aprovechando: el tempo en BPM, un índice de energía, uno de bailabilidad y uno de valencia o positividad del tono. Eso es lo que nos va a permitir recomendar por similitud de sonido, no solo por género.

Consultor: *¿Y del historial de reproducción? ¿Con qué frecuencia se consulta y qué necesitan saber exactamente?*

Gerente de Producto: Se consulta constantemente, prácticamente en tiempo real para el motor de recomendación, y de forma agregada una vez al día para reportes internos. Necesitamos saber qué usuario reprodujo qué canción, en qué momento exacto, desde qué tipo de dispositivo, y sobre todo cuántos segundos escuchó y si llegó a terminarla. Eso último es la señal más honesta de si le gustó o no.

Consultor: *¿Por qué es importante registrar el dispositivo desde el que se reproduce?*

Gerente de Producto: Porque el comportamiento de escucha varía mucho según el dispositivo: no es lo mismo alguien escuchando en bocina inteligente en casa que alguien en el celular en el transporte público. Nos interesa poder segmentar el abandono y las recomendaciones por tipo de dispositivo.

Consultor: *Sobre las playlists, ¿qué debe poder hacer un usuario con ellas?*

Gerente de Producto: Cada usuario puede crear sus propias playlists, ponerles nombre, y decidir si son colaborativas o no. Necesitamos poder guardar el orden exacto en que las canciones quedan dentro de la playlist, y también quién fue la persona que agregó cada canción y en qué momento, para tener trazabilidad.

Consultor: *Por último, ¿qué reportes esperan poder generar con este modelo?*

Gerente de Producto: Principalmente tres: canciones más escuchadas por rango de fecha y por perfil de energía/bailabilidad; usuarios con mayor tasa de abandono (reproducciones no completadas), segmentado por tipo de dispositivo, para ajustar el algoritmo de recomendación; y las playlists más activas por número de canciones y frecuencia de actualización.

3. Requerimientos del sistema

3.1 Requerimientos de datos

- Catálogo de artistas, álbumes y canciones, incluyendo atributos acústicos (BPM, energía, bailabilidad, valencia) por canción.
- Registro de usuarios con datos básicos de identidad (nombre, correo, fecha de registro, estatus activo).
- Catálogo de dispositivos usados por cada usuario para reproducir contenido.
- Log de reproducciones con usuario, canción, dispositivo, marca de tiempo, segundos escuchados e indicador de reproducción completa.
- Playlists con indicador de colaborativa, su creador, las canciones que contiene y la posición de cada una dentro de la lista.
- Registro de qué usuario agregó cada canción a una playlist y en qué fecha, para trazabilidad y para medir la frecuencia de actualización.

3.2 Requerimientos funcionales

- Registrar una reproducción cada vez que un usuario escucha una canción, capturando dispositivo y duración real de escucha.
- Calcular similitud entre canciones a partir de sus atributos acústicos para alimentar el motor de recomendación.
- Permitir que un usuario cree, edite y comparta playlists, ordenando las canciones dentro de ellas.
- Generar reportes de canciones más escuchadas (por rango de fecha y perfil de energía/bailabilidad), usuarios con mayor tasa de abandono segmentada por tipo de dispositivo, y actividad por playlist (número de canciones y frecuencia de actualización).
- Distinguir playlists colaborativas de playlists personales mediante un indicador booleano.

4. Modelo entidad-relación (DBML para dbdiagram.io)

El siguiente código en sintaxis DBML corresponde al diagrama entregado en modelo/diagrama-er.png, generado en dbdiagram.io a partir de este mismo esquema:

```
Table usuario {
  usuario_id int [pk, increment]
  nombre varchar
  correo varchar [unique]
  fecha_registro date
  activo boolean
}
```

```
Table artista {
  artista_id int [pk, increment]
  nombre varchar
  oyentes_mensuales int
  pais_origen varchar
}
```

```
Table album {
  album_id int [pk, increment]
  titulo varchar
  fecha_lanzamiento date
  artista_id int [ref: > artista.artista_id]
}
```

```
Table cancion {
  cancion_id int [pk, increment]
  titulo varchar
  duracion_seg int
  genero varchar
  album_id int [ref: > album.album_id]
  artista_id int [ref: > artista.artista_id]
  bpm int
  nivel_energia decimal
  bailabilidad decimal
  valencia decimal
}
```

```
Table dispositivo {
  dispositivo_id int [pk, increment]
  usuario_id int [ref: > usuario.usuario_id]
  tipo varchar
  sistema_operativo varchar
}
```

```
Table reproduccion {
  reproduccion_id int [pk, increment]
  usuario_id int [ref: > usuario.usuario_id]
  cancion_id int [ref: > cancion.cancion_id]
  dispositivo_id int [ref: > dispositivo.dispositivo_id]
  timestamp_inicio datetime
  segundos_escuchados int
  fue_completa boolean
}
```

```
Table playlist {
  playlist_id int [pk, increment]
  nombre varchar
  fecha_creacion date
  es_colaborativa boolean
  usuario_creador_id int [ref: > usuario.usuario_id]
}
```

```

Table lista_cancion {
  playlist_id int [ref: > playlist.playlist_id]
  cancion_id int [ref: > cancion.cancion_id]
  posicion int
  agregado_por int [ref: > usuario.usuario_id]
  fecha_agregado datetime
  indexes {
    (playlist_id, cancion_id) [pk]
  }
}

```

Las relaciones del modelo y sus cardinalidades, que también se representan en el diagrama, se resumen en la siguiente tabla:

Relación	Cardinalidad	Llave foránea	Descripción
artista – album	1 : N	album.artista_id	Un artista tiene varios álbumes; cada álbum pertenece a un solo artista.
artista – cancion	1 : N	cancion.artista_id	Un artista interpreta varias canciones; cada canción tiene un artista.
album – cancion	1 : N	cancion.album_id	Un álbum contiene varias canciones; una canción pertenece a un álbum (o a ninguno, si es un sencillo).
usuario – dispositivo	1 : N	dispositivo.usuario_id	Un usuario registra varios dispositivos; cada dispositivo es de un usuario.
usuario – reproduccion	1 : N	reproduccion.usuario_id	Un usuario genera muchas reproducciones.
cancion – reproduccion	1 : N	reproduccion.cancion_id	Una canción puede reproducirse muchas veces.
dispositivo – reproduccion	1 : N	reproduccion.dispositivo_id	Desde un dispositivo se realizan muchas reproducciones.
usuario – playlist	1 : N	playlist.usuario_creador_id	Un usuario crea varias playlists; cada playlist tiene un creador.
playlist – lista_cancion	1 : N	lista_cancion.playlist_id	Una playlist contiene varias canciones, una fila por cada una.
cancion – lista_cancion	1 : N	lista_cancion.cancion_id	Una canción puede aparecer en varias playlists.
usuario – lista_cancion	1 : N	lista_cancion.agregado_por	Un usuario agrega muchas canciones a playlists (trazabilidad de autoría).
playlist – cancion	N : M	vía lista_cancion	Relación muchos a muchos resuelta con la tabla puente lista_cancion.

5. Justificación del modelo

La entidad usuario se mantiene deliberadamente enfocada en datos de identidad (nombre, correo, fecha de registro, estatus), evitando mezclar en ella información de otra naturaleza que tendría

un ciclo de vida distinto. Las entidades artista, album y cancion replican la jerarquía natural del dominio musical (un artista tiene álbumes, un álbum tiene canciones), y cancion incorpora los cuatro atributos acústicos (bpm, nivel_energía, bailabilidad, valencia) exigidos explícitamente por el caso de negocio: sin estos campos, el motor de recomendación no tendría sobre qué calcular similitud de audio más allá del género. Además de la llave hacia album, cancion conserva una llave directa hacia artista: de esa forma se pueden representar sencillos o canciones que no pertenecen a ningún álbum, y se evita recorrer la tabla album cada vez que el motor de recomendación necesita relacionar una canción con su artista.

La entidad reproduccion es la tabla de hechos del sistema: concentra el log exacto que pide el cliente (usuario, canción, dispositivo, momento, segundos escuchados, si se completó). Se agregó dispositivo como entidad propia, en lugar de un campo de texto dentro de reproduccion, porque un mismo usuario puede tener varios dispositivos registrados y el negocio necesita analizarlos de forma independiente, tal como se confirmó en la entrevista: la segmentación de reportes de abandono por tipo de dispositivo es uno de los requerimientos explícitos.

Para resolver la relación muchos a muchos entre playlist y cancion se empleó la tabla puente lista_cancion, con llave primaria compuesta (playlist_id, cancion_id). Esta tabla conserva el orden de las canciones dentro de la playlist mediante el atributo posicion, y registra además quién agregó cada canción (agregado_por) mediante una llave foránea hacia usuario, lo cual es indispensable para tener trazabilidad de autoría dentro de una playlist, tal como pidió el cliente. Además, el atributo fecha_agregado registra el momento en que cada canción se incorporó a la playlist, lo que permite calcular la frecuencia de actualización de cada una y, con ello, generar el reporte de playlists más activas que solicitó el cliente.

En conjunto, el modelo resuelve la problemática planteada porque centraliza en una sola base de datos el catálogo musical con sus atributos de recomendación y el comportamiento real de escucha de cada usuario, conectando ambos mundos únicamente a través de las llaves foráneas estrictamente necesarias. El diseño queda abierto a extenderse en el futuro —por ejemplo, incorporando suscripciones, contenido de podcasts o relaciones de seguimiento entre usuarios— sin necesidad de modificar las tablas ya existentes, dado que cualquiera de esas extensiones se integraría como entidades adicionales relacionadas por llave foránea, sin alterar el esquema actual.